Задание: Написать программу с учётом следующего примера на C#, используя изученный метод транзакционной памяти и маневр для визуального показа с графикой при параллелизме желательно в мобильном приложении.

Мобильное приложение реализовать на .NET MAUI

Пример функционала 1:

Симуляция механизма STM на чистом C# (через CAS-операции) Самый близкий аналог STM, доступный в .NET «из коробки» для простых типов — это оптимистичная блокировка с использованием Interlocked.CompareExchange (механизм Compare-And-Swap / CAS).Вот пример безопасного изменения баланса банковского счета без использования ключевого слова lock:

```
csharpusing System;
using System.Threading;
using System.Threading.Tasks;

class Account
{
// Баланс хранится в приватном поле
private int _balance;

public int Balance => _balance;

public void DepositTransactional(int amount)
{
int currentBalance;
int newBalance;

do
{
// 1. Читаем текущее состояние («начало транзакции»)
currentBalance = _balance;

// 2. Вычисляем новое значение локально
newBalance = currentBalance + amount;

// Immitate some heavy work / context switch
Thread.SpinWait(10);

// 3. Пытаемся зафиксировать (Commit).
// CompareExchange проверяет: если _balance все еще равен currentBalance,
// то записывает newBalance. Возвращает то значение, которое БЫЛО в _balance.
}
// Если вернулось НЕ currentBalance, значит другой поток успел изменить _balance.
```

```
// Транзакция «откатывается» и цикл идет на новый круг (Retry).
while (Interlocked.CompareExchange(ref _balance, newBalance, currentBalance) != currentBalance);
}
}

class Program
{
static async Task Main()
{
var account = new Account();
Task[] tasks = new Task[100];

// 100 потоков одновременно пополняют счет на 10
for (int i = 0; i < tasks.Length; i++)
{
tasks[i] = Task.Run(() => account.DepositTransactional(10));
}

await Task.WhenAll(tasks);
Console.WriteLine($"Итоговый баланс: {account.Balance}"); // Всегда 1000
}
}
```

Пример функционала 2: Использование сторонних библиотек (Библиотека Nito.Transactions) Если вам нужно объединить в транзакцию изменение нескольких связанных объектов (сложное состояние), CAS-операции Interlocked не подойдут. Для этого используются специализированные STM-библиотеки. Одной из известных попыток сделать это в .NET была библиотека Nito.Transactions (или академические аналоги). Гипотетический пример, как выглядит работа со сложным STM в C# (через обертки объектов):

```
csharp// Объявляем транзакционные переменные
public class BankModel
{
public Transactional<int> AccountA = new Transactional<int>(100);
public Transactional<int> AccountB = new Transactional<int>(50);

public void Transfer(int amount)
{
// Запуск транзакции памяти
TxContext.ExecuteInTransaction(() =>
{
// Эти операции выполняются изолированно.
// Если в этот момент другой поток изменит AccountA или AccountB,
// весь этот блок прервется и начнется заново.
AccountA.Value -= amount;
AccountB.Value += amount;
});
```

```
}
}
```